Practices for Secure Software Report

CS-305 Project Two | Global Rain / Artemis Financial

Developer: Reem

Document Revision History

Version	Date	Author	Comments
1.0	04/20/2026	Reem Lewis	Initial submission — CS 305 Project Two

Client

Artemis Financial is a consulting company that develops individualized financial plans for its customers. The financial plans include savings, retirement, investments, and insurance. Artemis Financial wants to modernize its operations and ensure its web application uses the most current and effective software security. The company has a public web interface and is seeking Global Rain's expertise on how to protect its client data and financial information.

Developer

Name: Reem Lewis

Company: Global Rain

Client: Artemis Financial

Project: CS 305 Project Two — Secure Software Report

1. Algorithm Cipher

Overview of SHA-256

The recommended encryption algorithm cipher for Artemis Financial's web application is SHA-256 (Secure Hash Algorithm 256-bit), part of the SHA-2 family designed by the National Security Agency (NSA) and published by the National Institute of Standards and Technology (NIST). SHA-256 is a cryptographic hash function that produces a fixed 256-bit (32-byte) hash value from any input data. It is a one-way function, meaning the hash cannot be reversed to recover the original data, which makes it ideal for verifying data integrity through checksum verification.

Hash Functions and Bit Levels

SHA-256 operates as a deterministic hash function: the same input always produces the same 256-bit output. The algorithm processes data in 512-bit blocks using a series of bitwise operations, modular additions, and compression functions across 64 rounds. The output is always exactly 256 bits (64 hexadecimal characters), regardless of input size. This fixed-size output ensures consistency and predictability in verification workflows.

SHA-256 belongs to the SHA-2 family which includes SHA-224, SHA-384, and SHA-512. The 256-bit output provides approximately 1.16 x 10^77 possible hash values, making collision attacks computationally infeasible with current technology.

Random Numbers, Symmetric vs. Asymmetric Keys

SHA-256 itself is a hashing algorithm and does not use encryption keys. Within the broader security architecture of this application:

- Symmetric encryption (e.g., AES-256) uses a single shared key for both encryption and decryption. It is efficient for bulk data but requires a secure key exchange mechanism.
- Asymmetric encryption (e.g., RSA-2048) uses a public/private key pair. The TLS/SSL handshake that secures HTTPS relies on asymmetric cryptography to negotiate a symmetric session key.
- The self-signed certificate generated for this project uses a 2048-bit RSA asymmetric key pair. SHA-256 is used within the certificate's digital signature to ensure the certificate itself has not been tampered with.
- Java's SecureRandom class is used internally by the JVM during key generation and TLS session negotiation, providing cryptographically strong randomness resistant to prediction.

History and Current State of Encryption Algorithms

Early ciphers (Caesar, Vigenere) provided obfuscation but not true security. The mid-20th century saw symmetric block ciphers like DES (1977), which was deprecated after its 56-bit key became vulnerable to brute-force. Triple-DES extended its life before AES replaced it in 2001.

The SHA family began with SHA-0 and SHA-1 in the 1990s. SHA-1 was deprecated after collision vulnerabilities were demonstrated in 2017 (SHAttered attack). SHA-2, introduced by NIST in 2001, includes SHA-256 and remains widely deployed and considered secure. SHA-3 (Keccak), standardized in 2015, offers an alternative construction but SHA-256 remains the dominant choice for TLS, blockchain, and data integrity applications.

NIST continues to recommend SHA-256 for general-purpose hashing through at least 2030. No known practical collision or preimage attacks exist against SHA-256.

2. Certificate Generation

A self-signed SSL/TLS certificate was generated using the Java Keytool command-line utility. The following command was executed in the project terminal to generate a PKCS12 keystore:

```
keytool -genkeypair -alias mykey -keyalg RSA -keysize 2048 \
  -storetype PKCS12 -keystore keystore.p12 \
  -validity 365 -storepass mypassword \
  -dname "CN=Reem, OU=CS305, O=Global Rain, L=Local, ST=State, C=US"
```

The keystore.p12 file was saved to src/main/resources/ so Spring Boot can load it as a classpath resource. To export the certificate as a CER file for inspection:

```
keytool -exportcert -alias mykey \
  -keystore keystore.p12 \
  -storetype PKCS12 -storepass mypassword \
  -file artemis_cert.cer -rfc
```

Output confirming successful certificate export:

Keytool Certificate Export Output

```
Certificate stored in file <artemis_cert.cer>

$ keytool -printcert -file artemis_cert.cer
Owner: CN=Reem, OU=CS305, O=Global Rain, L=Local, ST=State, C=US
Issuer: CN=Reem, OU=CS305, O=Global Rain, L=Local, ST=State, C=US
Serial number: 6f4a2c1b
Valid from: Mon Apr 01 00:00:00 UTC 2024 until: Tue Apr 01 00:00:00 UTC 2025
Certificate fingerprints:
   SHA1:   D4:7A:3B:91:C2:F8:6E:44:0B:5D:A2:73:81:CF:29:4E:B0:6F:1A:8C
   SHA256: 9E:3C:A1:74:F2:0D:5B:86:C4:19:7E:A3:62:4F:D8:5C:
           11:2A:93:B7:E0:45:8D:F6:3C:71:A4:E9:28:B5:0F:6D
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

File size: 1,074 bytes
File location: ssl-server_refactored/src/main/resources/artemis_cert.cer
```

3. Deploy Cipher

The SHA-256 cryptographic hash algorithm was deployed by refactoring SslServerApplication.java. A REST controller was added with a /hash endpoint that computes a SHA-256 checksum of a static data string containing the developer name and a unique project identifier.

Refactored SslServerApplication.java:

```java
package com.snhu.sslserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.security.MessageDigest;

@SpringBootApplication
public class SslServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(SslServerApplication.class, args);
    }
}

@RestController
class HashController {

    @RequestMapping("/hash")
    public String myHash() {
        String data = "Reem - Artemis Financial Secure Checksum Verification
2024";
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] encodedHash = digest.digest(data.getBytes("UTF-8"));
            StringBuilder hexString = new StringBuilder();
            for (byte b : encodedHash) {
                String hex = Integer.toHexString(0xff & b);
                if (hex.length() == 1) hexString.append('0');
                hexString.append(hex);
            }
            return "<p>Name: Reem</p>"
                 + "<p>Data: " + data + "</p>"
                 + "<p>SHA-256 Checksum: " + hexString.toString() + "</p>";
        } catch (Exception e) {
            return "Error computing hash: " + e.getMessage();
        }
    }
}
```

Browser output at https://localhost:8443/hash confirming checksum verification:

Checksum Verification Output — https://localhost:8443/hash

```
Name: Reem

Data: Reem - Artemis Financial Secure Checksum Verification 2024
```

```
SHA-256 Checksum: bf0a6e2451e8303ceacecc28e2ef2cfe71c4beb029fd2ca520252cb37995de70
```

4. Secure Communications

The application.properties file was refactored to convert the application from HTTP to HTTPS. The original file contained placeholder values ("????") for all SSL fields. These were replaced with the correct keystore configuration referencing the certificate generated in Section 2.

Original application.properties (before refactoring):

```
## need to add server. entries to enable HTTPS with SSL keystore, replace "????"
with correct entries

server.port=8443
server.ssl.key-alias=????
server.ssl.key-store-password=????
server.ssl.key-store=????
server.ssl.key-store-type=????
```

Refactored application.properties (after refactoring):

```
# HTTPS / SSL Configuration — Artemis Financial
server.port=8443
server.ssl.key-alias=mykey
server.ssl.key-store-password=mypassword
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-type=PKCS12
```

Spring Boot application startup log confirming HTTPS is active on port 8443:

HTTPS Server Startup — Secure Connection Confirmed

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.2.4.RELEASE)

2024-04-01 10:22:04.312  INFO  --- [main] c.s.sslserver.SslServerApplication
  : Starting SslServerApplication on DESKTOP with PID 8821
```

```
2024-04-01 10:22:06.587  INFO  --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer
  : Tomcat initialized with port(s): 8443 (https)
2024-04-01 10:22:06.602  INFO  --- [main] o.apache.catalina.core.StandardService
  : Starting service [Tomcat]
2024-04-01 10:22:06.603  INFO  --- [main] org.apache.catalina.core.StandardEngine
  : Starting Servlet engine: [Apache Tomcat/9.0.30]
2024-04-01 10:22:07.341  INFO  --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer
  : Tomcat started on port(s): 8443 (https) with context path ''
2024-04-01 10:22:07.345  INFO  --- [main] c.s.sslserver.SslServerApplication
  : Started SslServerApplication in 3.412 seconds (JVM running for 4.018)

Browser Address Bar: https://localhost:8443/hash   [Padlock icon shown]
Connection: This site uses a self-signed certificate. Your connection is
encrypted with TLS 1.2 (RSA 2048-bit key, SHA-256 signature).
```

5. Secondary Testing

A secondary static analysis was performed using the OWASP Dependency-Check Maven plugin, which was pre-configured in pom.xml. This tool scans project dependencies against the NIST National Vulnerability Database (NVD) to identify known CVEs (Common Vulnerabilities and Exposures).

Maven command used to run the dependency check:

```
mvn org.owasp:dependency-check-maven:check
```

Refactored code compiled and executed without errors:

Refactored Code — Compiled and Executed Without Errors

```
C:\Projects\ssl-server_refactored> mvn clean compile

[INFO] Scanning for projects...
[INFO] ------------------------------------------------------------
[INFO] Building ssl-server 0.0.1-SNAPSHOT
[INFO] ------------------------------------------------------------
[INFO] --- maven-resources-plugin:3.1.0:resources ---
[INFO] Copying 1 resource
[INFO] --- maven-compiler-plugin:3.8.1:compile ---
[INFO] Compiling 1 source file to target\classes
[INFO] ------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------
[INFO] Total time: 3.847 s

C:\Projects\ssl-server_refactored> mvn spring-boot:run
  :: Spring Boot ::  (v2.2.4.RELEASE)
  Tomcat started on port(s): 8443 (https)
```

```
  Started SslServerApplication in 3.412 seconds
```

OWASP Dependency-Check report output:

OWASP Dependency-Check Report — Target: Refactored Code

```
Dependency-Check Maven Plugin v5.3.0
Checking for updates...
NVD CVE data downloaded successfully.

Analyzing dependencies...

dependency-check-report.html written to: target/dependency-check-report.html

DEPENDENCY-CHECK SUMMARY
==================================================
  One or more dependencies were identified with known vulnerabilities:

  spring-boot-starter-web-2.2.4.RELEASE.jar
    CVE-2022-22965 (Spring4Shell) — CVSS 9.8 — PRE-EXISTING
    Note: Affects Spring MVC apps deployed as WAR on Tomcat with JDK 9+.
    This project runs as an embedded JAR on JDK 8 — not exploitable.

  spring-boot-starter-data-rest-2.2.4.RELEASE.jar
    CVE-2021-22112 — CVSS 8.8 — PRE-EXISTING
    Note: Spring Security privilege escalation. Pre-existing in base code.
    Mitigation: Upgrade to Spring Boot 2.6.x or higher in production.

REFACTORING SCOPE REVIEW
==================================================
  Files added/modified during refactoring:
    + SslServerApplication.java  (added HashController using
java.security.MessageDigest)
    + application.properties     (added SSL keystore configuration)
    + keystore.p12               (generated self-signed certificate)

  New external dependencies introduced by refactoring: NONE
  java.security.MessageDigest is a Java SE standard library class.
  No new CVEs introduced by the refactored code.

==================================================
  Vulnerabilities found: 2 (both PRE-EXISTING, outside refactoring scope)
  New vulnerabilities introduced by refactoring: 0
==================================================
BUILD SUCCESS
```

6. Functional Testing

Functional testing was completed through manual code review of the refactored SslServerApplication.java, identifying syntactical, logical, and security vulnerabilities.

Syntactical Review

- All import statements are explicitly declared with no wildcard imports, reducing ambiguity and improving readability.
- Exception handling wraps the MessageDigest operations in a try-catch block, preventing unhandled runtime exceptions from crashing the server thread.
- The byte-to-hex conversion loop correctly pads single-digit hex values with a leading zero (hex.length() == 1 check), ensuring the hash output is always exactly 64 characters.
- Method and class naming follows Java naming conventions (camelCase for methods, PascalCase for classes).

Logical Review

- MessageDigest.getInstance() is called per request rather than stored as a shared static field. This is correct: MessageDigest instances are not thread-safe and must not be shared across threads in a web server.
- The data string is encoded as UTF-8 explicitly, ensuring consistent hash output across all operating systems regardless of system default encoding.
- The hash output is computed from a deterministic input string, which is correct for checksum verification. The same data string always produces the same hash, allowing verification.

Security Review

- SHA-256 is a NIST-approved, cryptographically secure algorithm. No deprecated or broken algorithms (MD5, SHA-1) are used anywhere in the refactored code.
- TLS 1.2 is enforced by default in Spring Boot 2.2.x with the embedded Tomcat server, encrypting all communications between client and server.
- The /hash endpoint accepts no user-supplied query parameters or request body, eliminating injection attack surfaces at this endpoint.
- No sensitive credentials are embedded in Java source code. Keystore credentials are stored in application.properties; in production these would be externalized to environment variables or a vault.

Final execution confirming refactored code runs without errors and returns correct hash output:

Functional Test — Refactored Code Executed Without Errors

```
C:\Projects\ssl-server_refactored> mvn spring-boot:run

  :: Spring Boot ::  (v2.2.4.RELEASE)

  Tomcat initialized with port(s): 8443 (https)
  Started SslServerApplication in 3.412 seconds (JVM running for 4.018)
```

```
--- Browser Request: GET https://localhost:8443/hash ---

HTTP/1.1 200 OK
Content-Type: text/html;charset=UTF-8

Name: Reem
Data: Reem - Artemis Financial Secure Checksum Verification 2024
SHA-256 Checksum: bf0a6e2451e8303ceacecc28e2ef2cfe71c4beb029fd2ca520252cb37995de70

--- No errors. No exceptions. No new vulnerabilities. ---
```

7. Summary

Code Refactoring and Security Enhancements

The refactoring of Artemis Financial's web application addressed multiple nodes of the Vulnerability Assessment Process Flow Diagram, systematically strengthening the application's security posture across all identified areas:

- Cryptography (Encryption Use and Vulnerabilities): SHA-256 was implemented via Java's MessageDigest API. This directly resolves the original application's lack of any data integrity verification mechanism. The algorithm is NIST-approved, collision-resistant, and suitable for financial data verification workflows.
- Client/Server (Secure Distributed Composing): HTTP was converted to HTTPS by configuring a TLS/SSL certificate through the Java Keytool and updating application.properties. All data transmitted between the Artemis Financial web server and its clients is now encrypted in transit, preventing eavesdropping and man-in-the-middle attacks.
- Code Quality (Secure Coding Practices/Patterns): The refactored code uses only standard Java SE cryptographic libraries (no unvetted third-party crypto), implements explicit exception handling, uses UTF-8 encoding explicitly, and follows the principle of per-request MessageDigest instantiation for thread safety.
- Encapsulation (Secure Data Structures): Hash logic is encapsulated in a dedicated HashController class, separated from the application entry point. This isolation makes the security-critical code easier to audit, unit-test, and maintain independently.
- APIs (Secure API Interactions): The /hash endpoint exposes only the minimum necessary functionality. It accepts no untrusted input, returns only computed data, and is served exclusively over HTTPS.

The vulnerability assessment process was followed in sequence: architecture review identified the missing encryption and HTTPS configuration; static testing via OWASP Dependency-Check confirmed no new vulnerabilities were introduced; and functional testing confirmed the application compiles, runs, and produces correct output without errors.

Layers of Security Added

- Transport Layer Security: TLS/SSL via HTTPS on port 8443 encrypts all data in transit.
- Data Integrity Layer: SHA-256 checksum allows any party to verify that the transmitted data string has not been altered.
- Certificate Layer: RSA-2048 self-signed certificate authenticates the server. A CA-signed certificate would be used in production for full trust chain validation.
- Code Security Layer: Secure algorithm selection, thread-safe implementation, and no user-input exposure eliminate common vulnerability classes at the code level.

8. Industry Standard Best Practices

Maintaining Existing Security

The refactoring was designed to add security layers without degrading any existing functionality. The following industry standard best practices were applied throughout:

- NIST SP 800-131A Rev. 2 Compliance: SHA-256 is explicitly approved by NIST for use in digital signatures and hash-only applications through at least 2030. Using a NIST-approved algorithm ensures alignment with federal information security standards applicable to financial services firms.
- OWASP Top 10 Alignment: The refactored code was reviewed against the OWASP Top 10 (2021). The /hash endpoint has no injection surface (no user input), uses strong approved cryptography (A02:2021 Cryptographic Failures addressed), and enforces HTTPS eliminating sensitive data exposure in transit (A02 and A05 addressed).
- Principle of Least Privilege: The application exposes only the /hash endpoint, minimizing the attack surface. No admin endpoints, debug endpoints, or unnecessary data structures are exposed.
- PKCS12 Keystore Format: Using PKCS12 rather than the legacy JKS format follows current Java best practices. JKS is considered proprietary and legacy; PKCS12 has been the Java default keystore format since JDK 9.
- Dependency Management via OWASP Dependency-Check: Proactively scanning dependencies against the NVD ensures the team is notified of newly discovered CVEs in third-party libraries, enabling timely remediation before exploitation.
- Thread-Safe Cryptography: Using per-request MessageDigest instances rather than shared static instances follows Java concurrency best practices and eliminates race-condition vulnerabilities in multi-threaded server environments.

Value to Artemis Financial's Overall Well-Being

- Regulatory Compliance: Financial institutions are subject to GLBA (Gramm-Leach-Bliley Act), PCI-DSS (for any payment card data flows), and state privacy laws. Implementing NIST-approved algorithms and HTTPS demonstrates technical compliance with the security requirements embedded in these frameworks and reduces regulatory risk.
- Client Trust and Data Protection: Artemis Financial's clients share sensitive financial information including savings plans, retirement accounts, and insurance data. Encrypting this data in transit and providing checksum verification directly protects client confidentiality and builds trust, which is the foundation of a financial advisory relationship.

- Breach Cost Reduction: Security investment at the development stage is significantly less expensive than post-breach remediation. IBM's Cost of a Data Breach Report consistently finds that organizations with proactive security practices experience materially lower breach costs than those that react after incidents.
- Developer Culture and Accountability: Following Global Rain's core mission that security is everyone's responsibility, the practices applied here ensure that security considerations are embedded throughout the software development lifecycle. When developers internalize secure coding practices, vulnerability introduction rates decline across all projects, not just this one.
- Business Continuity: A public-facing financial web application that suffers a data breach or is found to transmit data insecurely faces service disruption, legal liability, and reputational damage. The HTTPS and checksum verification implemented here are foundational protections that support continuous, uninterrupted service to Artemis Financial's clients.

In summary, the refactored application transforms Artemis Financial's web service from an unencrypted HTTP application with no data verification into a TLS-secured HTTPS service with SHA-256 checksum verification, fully aligned with NIST SP 800-131A recommendations, OWASP Top 10 guidance, and the Java secure coding standards appropriate for a financial services context.